

이상치와 중복 데이터

지울 것과 남길 것을 가르는 기준을 배웁니다

quantile IQR mask clip duplicated

2026년 8월 24일 (월) · 24일차 · 59차시

교안 16_01 이상치 개념과 사분위수 · 16_02 IQR 이상치 처리와 중복 데이터

국립금오공과대학교 컴퓨터공학부 박민호

교안	다룬 범위	상태
16_01 이상치 개념과 사분위수	노이즈와 신호 · 사분위수 · 실습 2~7	완주
16_02 IQR 이상치 처리와 중복	IQR · 제거·보정·변환 · 중복 · 실습 1~10	완주
16_02 실습 3 (박스플롯)	시각화 도구가 필요한 문제	건너뛸
다음 회차	17_01 선 그래프와 그래프 완성	8월 25일

이 책을 읽는 법

뱃지 · 코드블록 · 이전 회차와의 연결

1. 분류 뱃지

모든 항목 이름 앞에는 그것이 **무엇인지**를 알려주는 색 뱃지가 붙습니다. 이름만 외우면 나중에 반드시 헛갈립니다.

뱃지	무엇	판별 신호	오늘의 예
메서드	객체에 붙는 함수	점 뒤에 있고 괄호가 붙습니다	<code>.quantile()</code> <code>.clip()</code>
속성	객체가 가진 값	점 뒤에 있고 괄호가 없습니다	<code>.index</code> <code>.T</code>
키워드	함수에 넘기는 이름 붙은 인자	등호 왼쪽에 옵니다	<code>keep=</code> <code>subset=</code>
개념	문법이 아닌 사고 틀	코드로 쓸 수 없습니다	사분위수 · IQR · 노이즈와 신호
패턴	자주 쓰는 코드 흐름	여러 요소의 조합입니다	<code>Q1 - 1.5*IQR</code>

개념

3초 판별법. ① 앞에 점(.)이 있습니까? 없으면 함수나 키워드입니다. ② 점이 있다면 뒤에 괄호가 붙습니까? 붙으면 메서드, 안 붙으면 속성입니다. 오늘 자주 쓸 `df.drop_duplicates().reset_index(drop=True)`를 이 기준으로 읽으면 둘 다 메서드입니다. 반면 `report.T`는 괄호가 없으니 속성입니다.

2. 코드블록 읽는 법

회색 배경에 왼쪽 파란 바가 붙은 상자는 **파이썬 코드**입니다. 상자 아래 가로줄 밑에 초록색 ▶로 시작하는 줄은 **그 코드를 실제로 실행했을 때 나온 출력**입니다. 이 책의 모든 출력은 실제 실행 결과를 옮긴 것입니다.

예시

```
import pandas as pd
```

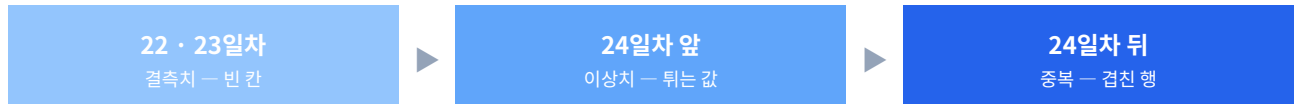
```
df = pd.read_csv("Data/16_diecasting.csv", encoding="utf-8")
print(df["사이클타임"].quantile([0.25, 0.5, 0.75]))
```

```
▶ 0.25    20.800
▶ 0.50    22.600
▶ 0.75    35.925
▶ Name: 사이클타임, dtype: float64
```

본문에서 이렇게 회색으로 칠해진 글자는 코드 조각입니다. **굵은 글씨**는 꼭 붙잡아야 할 문장입니다.

3. 이전 회차와의 연결

지난 시간(23일차)에 **결측치 단원을 끝냈습니다**. 오늘은 새 단원인 **이상치**로 넘어갑니다. 그리고 하루 만에 이상치 단원을 통째로 끝냈습니다. 실습이 열 개나 되어 분량이 많았습니다.



연결

지난 시간에 배운 "**확인 → 판단 → 처리 → 재확인**" 고리가 오늘 두 번 더 반복됩니다. 이상치에서 한 번, 중복에서 한 번입니다. **도구 이름만 바꿀 뿐 사고의 틀은 똑같습니다**. `isna()` 자리에 `quantile()`이 들어가고 `duplicated()`가 들어갈 뿐입니다. 그리고 처리 단계에서는 지난 시간에 배운 `fillna()`가 그대로 다시 쓰입니다.

CONTENTS

목차

오늘 배우는 것을 한눈에

장	제목	무엇을 배우나
CH 0	오늘의 지도	결측치와 이상치는 무엇이 다른가
STEP 1	이상치란 무엇인가	노이즈와 신호 · 지우면 안 되는 이유
STEP 2	분포를 숫자로 읽기	평균-중앙값 격차 · 사분위수 · describe
STEP 3	IQR과 1.5배 규칙	경계 공식 · mask로 골라내기
STEP 4	이상치 처리 세 가지	제거 · clip 보정 · NaN 변환 후 채움
STEP 5	중복 데이터	duplicated · keep · drop_duplicates
STEP 6	융접 데이터 종합 정제	탐색부터 저장까지 한 흐름
부록 A	치트시트	오늘 나온 모든 도구를 한 표에
부록 B	오류·함정 사전	제출 코드 교정 · 자주 걸리는 함정
부록 C	실습 하이라이트와 다음 시간	핵심 결론 정리 · 시각화 단원 예고

개념

오늘의 한 문장. **"이상치를 발견했을 때 물어야 할 것은 지울까 말까가 아니라 노이즈인가 신호인가다."** 설비 고장은 언제나 평소와 다른 상태에서 일어납니다. 그 순간이 곧 이상치입니다. 다 지워 버리면 **고장을 알려 주던 단서까지 함께 사라집니다.**

오늘의 지도

결측치와 이상치는 무엇이 다른가

0-1. 데이터가 지저분한 세 가지 방식

지금까지 배운 것과 오늘 배울 것을 한 표에 놓으면 관계가 분명해집니다. **값이 비었거나, 값이 튀었거나, 행이 겹쳤거나.** 셋이 전부입니다.

문제	증상	찾는 법	처리	언제
결측치	값이 비어 있음	<code>isna()</code>	<code>dropna / fillna</code>	22·23일차
이상치	값은 있는데 동떨어짐	<code>quantile</code> → IQR	<code>~mask / clip</code>	오늘 앞
중복	행이 통째로 반복됨	<code>duplicated()</code>	<code>drop_duplicates()</code>	오늘 뒤

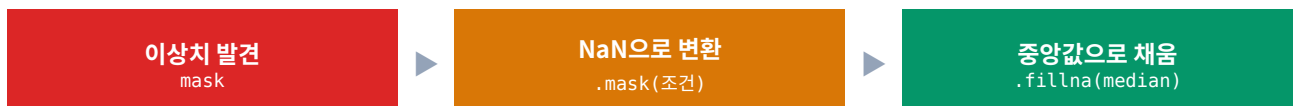
개념

셋의 결정적 차이는 **찾는 법**입니다. 결측은 `isna()`로 정확히 잡히지만, **이상치는 `isna()`로 절대 못 찾습니다.**

값이 비어 있지 않고 멀쩡한 숫자로 채워져 있기 때문입니다. 250도라는 값은 `isna()` 입장에서는 아무 문제가 없는 값입니다. 그래서 이상치는 **분포를 계산해서** 찾아야 하고, 그 계산 도구가 오늘 배울 사분위수와 IQR입니다.

0-2. 결측치와 이상치는 처리에서 만난다

찾는 법은 완전히 다르지만 **처리 단계에서는 서로 손을 잡습니다.** 이상치 처리의 한 방법이 **그 값을 일단 결측으로 바꾼 뒤 채우는 것**이기 때문입니다.



지난 시간에 배운 `fillna()`가 오늘 이상치 처리에 그대로 쓰입니다. **믿을 수 없는 값을 일단 비우고 주변 값으로 메우는 것**이지요. 그래서 두 단원을 붙여서 배우는 것입니다.

0-3. 오늘 다루는 데이터

파일	행 × 열	공정	어디에 쓰나
16_diecasting.csv	202 × 7	다이캐스팅 주조	STEP 1~5 대부분
16_welding.csv	162 × 6	저항 점용접	STEP 6 종합 정제
sales_data.csv	8 × 6	(도구 비교용 샘플)	부록 C 보너스

설비 적용

다이캐스팅은 녹인 금속을 고압으로 금형에 주입해 부품을 만드는 주조 방식입니다. 한 번의 주입을 **샷**이라 부릅니다. 실린더압력은 금속을 밀어 넣는 유압, 주조압력은 금형 안에 걸리는 압력, 비스킷두께는 주입 후 남은 금속 덩어리의 두께입니다. 사이클타임은 한 샷에 걸린 시간이고, **정상이면 20~35초** 부근입니다. 상태 컬럼은 0이 정상, 1이 문제 있음입니다.

데이터의 성격을 미리 알아 두면 뒤의 모든 계산이 잘 읽힙니다. **202행 중 상태가 1인 행이 20건**입니다. 그리고 **결측이 다섯 컬럼에 각각 14건씩** 있습니다. 이 14라는 숫자가 뒤에서 두 번 함정으로 등장합니다.

오늘 데이터 첫인상

```
import pandas as pd
```

```
df = pd.read_csv("Data/16_diecasting.csv", encoding="utf-8")
print(df.shape)
print(df.isna().sum())
print("상태 분포:", df["상태"].value_counts().to_dict())
```

```
▶ (202, 7)
▶
▶ 샷    0
▶ 실린더압력  14
▶ 주조압력    14
▶ 사이클타임  14
▶ 비스킷두께  14
▶ 형체력     14
▶ 상태      0
▶ dtype: int64
▶
▶ 상태 분포: {0: 182, 1: 20}
```

주의

결측 14건이 다섯 컬럼에서 똑같습니다. 22일차에 배운 대로 이런 패턴은 같은 행에서 함께 빈다는 신호입니다. 실제로 확인해 보면 14건이 전부 같은 행입니다. 즉 14개 샷은 샷 번호와 상태만 기록되고 측정값이 통째로 없습니다. 이 사실을 모르고 이상치를 걸러 내면 뒤에서 숫자가 어긋납니다.

0-4. 오늘 등장하는 새 용어

용어	한 줄 정의	왜 필요한가
사분위수	데이터를 크기순 네 등분하는 세 경계값 (Q1·Q2·Q3)	이상치에 흔들리지 않는 분포 지표
IQR	Q3 - Q1 — 가운데 절반이 차지하는 폭	그 데이터 자신의 퍼짐을 잣대로 삼기
노이즈성 이상치	설비 상태와 무관하게 잘못 들어온 잡음	정리 대상
신호성 이상치	설비가 실제로 평소와 다르게 동작한 값	보존 대상
원저화	경계 밖 값을 경계값으로 눌러 평탄화	<code>clip()</code>
완전 중복	모든 컬럼 값이 똑같은 행	하나만 남기면 됨
부분 중복	핵심 컬럼만 같은 행	기준 컬럼을 사람이 정해야 함

STEP 1

이상치란 무엇인가

노이즈와 신호 · 지우면 안 되는 이유

오늘 수업에서 **강사님이 가장 오래, 가장 힘주어 설명하신 부분**입니다. 코드는 몇 줄 안 되지만 사고방식이 핵심이고, 뒤에 나올 모든 처리 판단이 여기서 갈립니다.

1-1. 이상치의 정의

개념 이상치 (outlier) 다른 값들과 크게 동떨어진 값

정의 다른 값들과 **크게 동떨어진 값**입니다. 모터 온도 72·75·78도 사이에 갑자기 찍힌 250도, 진동이 평소 3인데 한 번 찍힌 12 같은 값입니다.

왜 쓰나 설비 데이터에서 이상치는 **그 자체가 고장 신호일 수 있습니다**. 다른 분야에서는 그냥 잡음이지만, 예지보전에서는 우리가 찾으려는 바로 그 대상일 수 있습니다.

응용 결측치와 나란히 놓고 보면 차이가 분명해집니다.

개념

표에서 가장 중요한 줄은 **"찾는 법"**입니다. 이상치는 값이 비어 있지 않고 멀쩡한 숫자로 채워져 있어서 `isna()`가 절대 잡지 못합니다. **분포를 계산해야만 찾을 수 있습니다**.

구분		
상태	값이 비어 있음	값은 있으나 동떨어짐
표시	NaN	채워진 숫자 (예: 250)
찾는 법	<code>isna()</code>	<code>isna()</code>로 못 찾음 — 분포 계산 필요
처리	제거 또는 대체	제거 · 보정 · NaN 변환 후 채움

1-2. 노이즈인가 신호인가

오늘의 핵심 질문입니다. 강사님 표현을 그대로 옮기면 **"노이즈의 이상치**하고 **신호의 이상치**"를 가르는 일입니다.

구분		
정체	설비 상태와 무관하게 잘못 들어온 잡음	설비가 실제로 평소와 다르게 동작한 값

구분		
예	온도 -5도 — 물리적으로 불가능	온도 95도 — 가능하지만 평소와 다름
원인	센서 노후·접촉 불량·통신 끊김·단위 혼동	과부하·베어링 마모·냉각 실패
처리	정리 대상	보존 대상

개념

가르는 첫 번째 잣대는 **물리적 가능성**입니다. 물리적으로 불가능하면 **노이즈**, 가능한데 평소와 다르면 **신호**입니다. 온도 -5도로 정상 가동되는 설비는 없으니 그것은 노이즈입니다. 반면 95도는 가능한 값이니 **진짜 과열일 수 있습니다**. 그리고 이 판단에는 **설비를 아는 도메인 지식**이 반드시 필요합니다.

강사님이 이 대목에서 강조하신 것은 **혼자 판단하지 말라**는 점이었습니다. "**현장하고 커뮤니케이션 하면 따져 볼 수 있게 돼요. 여러분이 임의로 할 것도 아니고, AI한테 물어보고 네가 알아서 노이즈인지 신호인지 따져 봐 이렇게 될 것도 아니에요.**" 데이터만 봐서는 알 수 없는 것이 있고, 그 답은 **현장에 있습니다**.

1-3. 화재경보기 이야기

강사님이 이 개념을 설명하려고 드신 비유입니다. 오늘 수업에서 가장 인상 깊었던 대목이라 그대로 옮깁니다.

비유

"화재경보기가 울리는 케이스는 단순하죠. 경보기 입장에서 평소에 잡히는 수치보다 이상 값이 잡히면 울려 버리는 거잖아요. 정상 범위를 벗어났다 하는 이상한 값이 잡히는 순간 화재경보를 울린다고요."

"그러니까 화재경보기가 울린다고 해서 소리를 꺼 버리면 안 되고, 진짜 불이 크게 났을 때 아무도 알 수 없으니까."

이상치를 무조건 지우는 것이 **경보기를 꺼 버리는 일**과 같다는 뜻입니다. 강사님은 지하 주차장의 가스 감지기 예도 드셨습니다. **경보가 시끄럽다고 꺼 놓으면 진짜 화재 때 아무도 모릅니다**.

설비 적용

데이터로 옮기면 이렇습니다. **고장은 언제나 평소와 다른 상태에서 발생합니다. 그 순간이 곧 이상치**입니다.

이상치를 전부 지우고 모델을 학습시키면, 그 모델은 **정상 상태만 본 모델**이 됩니다. 고장을 예측하라고 만든 모델이 고장을 한 번도 본 적이 없는 것입니다. 이것이 예지보전에서 이상치 처리를 조심해야 하는 이유입니다.

1-4. 처리는 3단계 — 판단을 건너뛰지 않기



단계	하는 일	누가	오늘 배우는 도구
탐색	동떨어진 값의 위치를 찾아냄	코드가 자동으로	quantile → IQR → mask
판단	노이즈인지 신호인지 결정	사람 — 도메인 지식	없음 — 코드로 못 함
처리	남길지 바꿀지 실행	코드가	~mask · clip · mask+fillna

주의

가장 중요한 것은 2단계를 생략하지 않는 것입니다. 그런데 2단계는 코드로 할 수 없는 유일한 단계입니다. IQR이 후보를 짚어 주면 사람이 하나씩 들여다보고 결정해야 합니다. 그래서 교안은 2단계를 "가장 중요하고 가장 어려운" 단계라고 적었습니다. 오늘 배우는 코드는 전부 1단계와 3단계입니다.

그리고 판단한 내용을 기록으로 남겨야 합니다. 무엇을 왜 노이즈로 판단했고 무엇을 신호로 보존했는지 적어 두지 않으면, 나중에 결과가 이상할 때 어디서부터 되짚어야 할지 알 수 없습니다. 이상치 처리는 한 번 하고 끝나는 일이 아니라 데이터가 쌓일 때마다 반복되는 관리 활동입니다.

STEP 2

분포를 숫자로 읽기

평균-중앙값 격차 · 사분위수 · describe

IQR을 쓰려면 **분포를 숫자로 읽는 법**이 먼저입니다. 지난 시간에 배운 평균과 중앙값이 여기서 이상치 탐지 도구로 승격됩니다.

2-1. 실습 2 — 범위, 그리고 범위의 약점

Practice/16_01_example.py — 실습 2

```
df = pd.read_csv("Data/16_diecasting.csv", encoding="utf-8")
```

```
print(df["실린더압력"].min())
```

```
print(df["실린더압력"].max())
```

```
print(df["실린더압력"].max() - df["실린더압력"].min())
```

```
▶ 108.0
```

```
▶ 265.0
```

```
▶ 157.0
```

범위 = 최댓값 - 최솟값. 퍼진 폭을 한 숫자로 표현하는 가장 간단한 방법입니다. 그런데 치명적인 약점이 있습니다.

개념

범위는 양 끝 두 값만으로 계산됩니다. 그래서 **이상치 하나에 완전히 휘둘립니다.** 300도짜리 오측정 하나만 섞여도 범위가 확 커집니다. 흔들리는 자로 흔들리는 대상을 재는 셈입니다. 그래서 **가운데를 보는 사분위수와 IQR**로 넘어갑니다.

관점		
중심	평균 mean	중앙값 median
퍼짐	범위 range · 표준편차 std	IQR
왜	양 끝값 · 모든 값이 계산에 직접 들어감	가운데 절반만 보므로 끝값 무시

2-2. 실습 3 · 4 — 정렬과 평균-중앙값 격차

이상치를 찾는 가장 원시적인 방법은 **정렬해서 눈으로 보는 것**입니다. 그리고 그 다음이 **평균과 중앙값을 나란히 놓는 것**입니다.

Practice/16_01_example.py — 실습 3

```
# 사이클타임 열을 기준으로 내림차순 정렬
s_sorted = df.sort_values("사이클타임", ascending=False)
print(s_sorted[["샷", "사이클타임", "상태"]].head(6))
```

▶ 샷 사이클타임 상태

```
▶ 199 200 6170.0 1 ← 설비 잼
▶ 180 181 652.3 1
▶ 146 147 125.9 0
▶ 181 182 93.1 1
▶ 190 191 91.2 1
▶ 195 196 76.3 1
```

6170초는 약 1시간 43분입니다. 정상 사이클이 20~35초인 공정에서 이 값이 나왔다면 **설비 잼(jam)**, 즉 금속이 굳어 붙어 라인이 멈춘 상황으로 봐야 합니다. 상태 컬럼도 1로 찍혀 있어 이 해석을 뒷받침합니다.

주의

상위 6건 중 5건이 상태 1입니다. 그런데 세 번째 125.9초는 상태가 0, 즉 정상으로 기록되어 있습니다. 이런 값이 가장 판단하기 어렵습니다. 기록상으로는 정상인데 사이클타임이 정상 범위의 네 배입니다. 기록이 잘못됐을 수도 있고, 실제로 정상인데 뭔가 특별한 상황이었을 수도 있습니다. 1장에서 말한 대로 이 판단은 코드가 아니라 현장입니다.

패턴 평균 - 중앙값 격차 이상치가 있는지 알려주는 신호

정의 평균은 극단값에 끌려가고 중앙값은 흔들리지 않습니다. 그래서 두 값의 격차 자체가 이상치의 신호입니다.

```
print(df["사이클타임"].agg(["mean", "median"]))

# 상태가 정상인 행만 골라 평균을 다시 계산
df_ok = df[df["상태"] == 0]
print("정상만의 평균:", df_ok["사이클타임"].mean().round(2))
```

```
▶ mean      64.75
▶ median     22.60
▶ Name: 사이클타임, dtype: float64
▶
▶ 정상만의 평균: 27.67
```

왜 쓰나 평균 64.75초와 중앙값 22.6초는 세 배 가까이 차이가 납니다. 이 격차 자체가 위쪽에 극단값이 있다는 증거입니다. 히스토그램을 그리지 않고도 알 수 있습니다.

응용 상태 1인 행을 빼고 다시 평균을 내면 **27.67초**가 나옵니다. 중앙값 22.6에 훨씬 가까워졌습니다. **극단값 20건이 평균을 37초나 끌어올리고 있었던 것입니다.**

```
# 이상 상태 행이 평균을 얼마나 끌어올렸나
print("전체 평균 :", round(df["사이클타임"].mean(), 2))
print("정상만 평균:", round(df_ok["사이클타임"].mean(), 2))
print("차이   :", round(df["사이클타임"].mean() - df_ok["사이클타임"].mean(), 2), "초")
print("상태 1 건수:", (df["상태"] == 1).sum(), "/", len(df))
```

```
▶ 전체 평균 : 64.75
▶ 정상만 평균: 27.67
▶ 차이   : 37.08 초
▶ 상태 1 건수: 20 / 202
```

개념

202건 중 20건, 10%가 평균을 두 배 넘게 부풀렸습니다. 이것이 어제 배운 "평균은 극단값에 끌려간다"의 실제 사례입니다. 데이터를 받으면 **평균과 중앙값을 항상 나란히 확인하는 작은 습관 하나**가 큰 오해를 막습니다.

2-3. 실습 5 — quantile로 사분위수 구하기

매서드 `.quantile()` 지정한 비율 지점의 값 구하기

정의 데이터를 크기순으로 줄 세웠을 때 **그 비율 지점에 있는 값**을 돌려줍니다. 괄호에는 **퍼센트를 100으로 나눈 소수**를 넣습니다.

형태 `df["열"].quantile(0.25)` | `df["열"].quantile([0.25, 0.5, 0.75])`

```
print(df["실린더압력"].quantile(0.25))    # Q1
print(df["실린더압력"].quantile(0.50))    # Q2
print(df["실린더압력"].median())          # 중앙값과 같은가?
print(df["실린더압력"].quantile(0.75))    # Q3
```

```
▶ 215.75
▶ 218.0
▶ 218.0   ← Q2 = 중앙값, 정확히 일치
▶ 265.0
```

왜 쓰나 Q2는 중앙값과 정확히 같습니다. 사분위수는 중앙값 사고방식을 확장한 것이기 때문입니다. 중앙값이 데이터를 절반으로 나누는 선이라면, 사분위수는 **네 등분하는 세 개의 선**입니다.

응용 리스트로 넘기면 한 번에 구해 Series로 돌려줍니다. `.tolist()`로 파이썬 리스트로 꺼낼 수도 있습니다.

```
q = df["주조압력"].quantile([0.25, 0.5, 0.75])
print(q)
print(q.tolist())
```

```
▶ 0.25      595.0
▶ 0.50     1037.0
▶ 0.75     1052.0
▶ Name: 주조압력, dtype: float64
▶
▶ [595.0, 1037.0, 1052.0]
```

개념

quantile()은 **결측을 자동으로 제외**하고 계산합니다. 그리고 **정렬도 알아서 합니다**. 강사님 코드 주석대로 sort_values()를 미리 할 필요가 없습니다.

주의

괄호에 넣는 숫자는 **반드시 0과 1 사이 소수**입니다. quantile(25)처럼 퍼센트를 넣으면 오류가 납니다. 0.9를 넣으면 90백분위수, 0.99면 상위 1% 경계가 됩니다. **Q1은 0.25, Q2는 0.5, Q3는 0.75**로 외우십시오.

개념

주조압력의 사분위수를 읽어 보십시오. **Q1에서 Q2까지는 442만큼 벌어져 있는데, Q2에서 Q3까지는 15밖에 안 됩니다**. 강사님 코드의 주석이 이 뜻을 정확히 풀어 놓았습니다. "데이터가 1000~1050에 몰려 있고 아래쪽으로 꼬리가 긴 분포. 정상 주조압력은 1000 언저리인데 저압 이상이 꽤 섞여 있다는 뜻." 사분위수 세 개만으로 분포의 모양을 읽어 낸 것입니다.

2-4. 실습 6 — describe로 격차 큰 컬럼 찾기

컬럼이 여러 개일 때 어느 것부터 봐야 할까요. **평균과 중앙값의 격차가 큰 컬럼**부터입니다. 그 계산을 표로 자동화하는 것이 실습 6입니다.

패턴 describe().T + 격차 열 이상치 의심 컬럼을 자동으로 줄 세우기

정의 describe() 결과를 전치(.T)해 컬럼을 행으로 눕힌 뒤, **평균과 중앙값의 차이**를 새 열로 계산합니다.

형태 df[cols].describe().T → ["mean"] - ["50%"]

```
report = df[["실린더압력", "주조압력", "사이클타임", "비스킷두께", "형체력"]].describe().T

# 새로운 컬럼 이름을 언급하면 그 자리에 추가된다
report["격차"] = (report["mean"] - report["50%"]).abs()

print(report.sort_values("격차", ascending=False)[["mean", "50%", "max",
"격차"]].round(2))
```

	mean	50%	max	격차
▶ 주조압력	848.70	1037.0	1159.0	188.30
▶ 형체력	303.11	258.0	384.0	45.11
▶ 사이클타임	64.75	22.6	6170.0	42.15
▶ 실린더압력	234.31	218.0	265.0	16.31
▶ 비스킷두께	14.17	13.0	27.0	1.17

왜 쓰나 컬럼이 다섯 개면 눈으로 봐도 되지만 수십 개면 불가능합니다. 격차 열을 만들어 정렬하면 어느 컬럼부터 손볼지 순서가 자동으로 정해집니다.

응용 .T는 전치(transpose)입니다. describe()는 통계 항목이 행, 컬럼이 열인 표를 주는데, .T로 뒤집으면 컬럼이 행이 되어 컬럼별로 읽기 편해집니다. 지난 시간에 배운 agg와 마찬가지로 표의 방향을 다루는 감각입니다.

```
# .T 없이 보면 이렇게 나온다 — 읽기 불편하다
print(df[["실린더압력", "사이클타임"]].describe().round(2))
```

	실린더압력	사이클타임
▶ count	188.00	188.00
▶ mean	234.31	64.75
▶ std	32.12	450.13
▶ min	108.00	20.60
▶ 25%	215.75	20.80
▶ 50%	218.00	22.60
▶ 75%	265.00	35.92
▶ max	265.00	6170.00

개념

report["격차"] = ...처럼 없던 컬럼 이름을 왼쪽에 쓰면 그 자리에 새 열이 만들어집니다. 제출 코드의 주석이 이 문법을 정확히 짚었습니다. "새로운 컬럼 이름을 언급하면 추가가 된다."

주의

.abs()를 붙인 이유는 **격차가 음수일 수도 있기 때문**입니다. 주조압력은 평균이 중앙값보다 낮아 -188.30이 나오는데, 절댓값으로 바꿔야 크기 순으로 줄 세울 수 있습니다. 21일차에 상관계수를 절댓값으로 정렬한 것과 같은 이유입니다.

결과를 읽어 봅시다. **주조압력의 격차가 188.30으로 압도적 1위**입니다. 평균 848.70인데 중앙값이 1037.0입니다. **평균이 중앙값보다 낮다**는 것은 **아래쪽으로 극단값이 있다**는 뜻입니다. 저압 이상이 여러 건 섞여 평균을 끌어내린 것입니다.

주의

그런데 **사이클타임의 격차가 42.15로 3위**인 점에 주의해야 합니다. 최댓값이 **6170**인데도 순위가 낮습니다. 왜 그럴까요. **격차는 절대값이라 컬럼의 단위 크기에 좌우되기 때문**입니다. 주조압력은 1000 단위라 격차 188이 나오지만, 사이클타임은 20 단위라 6170짜리 극단값이 있어도 격차가 42에 그칩니다. **비율로 보면 사이클타임이 훨씬 심각합니다**. 격차 42.15를 중앙값 22.6으로 나누면 186%이고, 주조압력은 188.30 나누기 1037로 18%에 불과합니다.

단위를 정규화해서 다시 보기

격차를 중앙값으로 나눠 상대 격차로 다시 줄 세우기

```
report["상대격차%"] = (report["격차"] / report["50%"] * 100).round(1)
print(report.sort_values("상대격차%", ascending=False)[["mean", "50%",
"격차", "상대격차%"]].round(2))
```

	mean	50%	격차	상대격차%
▶ 사이클타임	64.75	22.6	42.15	186.5
▶ 주조압력	848.70	1037.0	188.30	18.2
▶ 형체력	303.11	258.0	45.11	17.5
▶ 비스킷두께	14.17	13.0	1.17	9.0
▶ 실린더압력	234.31	218.0	16.31	7.5

순위가 완전히 뒤집혔습니다. **사이클타임이 186.5%로 압도적 1위**가 됩니다. 실제로 가장 심각한 컬럼이 맞습니다. 단위가 다른 컬럼을 절대값으로 비교하면 큰 단위 컬럼이 유리해집니다. 실습이 요구한 대로 절대 격차로 푸는 것도 맞지만, 실무에서는 이 한 줄을 더 붙이는 편이 안전합니다.

2-5. 실습 7 — 여러 컬럼의 가운데 절반 폭

마지막 실습은 다음 시간의 IQR을 미리 손으로 계산해 보는 것입니다. Q3에서 Q1을 뺀 값이 곧 IQR입니다.

세 컬럼의 사분위수를 한 번에

```
df_q = df[["실린더압력", "사이클타임", "비스킷두께"]].quantile([0.25, 0.50, 0.75])
print(df_q)
```

0.75가 index label인 행에서 0.25 행을 뺀다

```
print(df_q.loc[0.75] - df_q.loc[0.25])
```

```

▶ 실린더압력 사이클타임 비스킷두께
▶ 0.25  215.75  20.800  11.0
▶ 0.50  218.00  22.600  13.0
▶ 0.75  265.00  35.925  17.0
▶
▶ 실린더압력  49.250
▶ 사이클타임  15.125
▶ 비스킷두께   6.000
▶ dtype: float64
```

개념

`df_q.loc[0.75]`에 주목하십시오. **0.75는 행 번호가 아니라 인덱스 라벨**입니다. `quantile([0.25, 0.5, 0.75])`가 만든 표는 인덱스가 0.25·0.50·0.75라는 소수입니다. 그래서 위치 기반인 `iloc` 대신 **라벨 기반인 `loc`를 써야 합니다.** 앞서 배운 `loc`와 `iloc`의 차이가 여기서 실전으로 쓰입니다.

그리고 **Series끼리 뺄셈**이 됩니다. `df_q.loc[0.75]`도 Series, `df_q.loc[0.25]`도 Series이고, 둘 다 인덱스가 컬럼 이름이라 **같은 이름끼리 짝을 맞춰 빼** 줍니다. 넘파이에서 배운 벡터 연산이 판다스에서도 그대로 통합니다.

설비 적용

폭이 좁을수록 **안정적인 컬럼**입니다. 비스킷두께는 폭이 6.0으로 가장 좁아 공정이 잘 관리되고 있고, 실린더압력은 49.25로 가장 넓습니다. 다만 **단위가 다르므로 숫자를 직접 비교하면 안 됩니다.** 실린더압력은 200 단위, 비스킷두께는 13 단위입니다. 중앙값 대비로 보면 실린더압력 22.6%, 사이클타임 66.9%, 비스킷두께 46.2%로 **사이클타임이 가장 불안정**합니다.

STEP 3

IQR과 1.5배 규칙

경계 공식 · mask로 골라내기

앞 장에서 손으로 계산한 $Q3 - Q1$ 에 정식 이름이 붙습니다. **IQR — Inter-Quartile Range**, 사분위 범위입니다.

3-1. IQR — 가운데 절반의 폭

개념 IQR $Q3 - Q1$ — 가운데 절반이 차지하는 폭

정의 Q3에서 Q1을 뺀 값입니다. 데이터를 크기순으로 줄 세웠을 때 가운데 50%가 차지하는 폭을 뜻합니다.

형태 $IQR = Q3 - Q1$

```
q1 = df["실린더압력"].quantile(0.25)
q3 = df["실린더압력"].quantile(0.75)
iqr = q3 - q1
print(f"Q1: {q1}, Q3: {q3}")
print(f"IQR: {iqr}")
```

```
▶ Q1: 215.75, Q3: 265.0
▶ IQR: 49.25
```

왜 쓰나 범위와 달리 이상치에 안 흔들립니다. 범위는 양 끝 두 값만 보지만 IQR은 가운데 절반만 봅니다. 6170초짜리 값이 하나 들어와도 Q1과 Q3는 거의 그대로입니다.

응용 왜 데이터마다 다른 잣대를 쓸까요. 모든 컬럼에 똑같은 고정 숫자를 쓸 수 없기 때문입니다. 온도는 70~80이 정상이고 압력은 1000~1050이 정상인데, 같은 기준으로 잴 수 없습니다. 그래서 그 데이터 자신의 퍼짐(IQR)을 잣대로 삼습니다.

```
# 같은 코드가 컬럼마다 자기 잣대를 만든다
for c in ["실린더압력", "사이클타임", "비스킷두께"]:
    q1, q3 = df[c].quantile(0.25), df[c].quantile(0.75)
    print(f"{c:6s} Q1={q1:8.2f} Q3={q3:8.2f} IQR={q3-q1:7.2f}")
```

```
▶ 실린더압력 Q1= 215.75 Q3= 265.00 IQR= 49.25
▶ 사이클타임 Q1= 20.80 Q3= 35.93 IQR= 15.13
▶ 비스킷두께 Q1= 11.00 Q3= 17.00 IQR= 6.00
```

개념

이것이 IQR의 최대 강점입니다. 수백 개 컬럼도 같은 규칙으로 한 번에 적용할 수 있습니다. 컬럼마다 사람이 정상 범위를 지정하는 것은 현실적으로 불가능하지만, IQR은 반복문 한 줄이면 끝납니다.

3-2. 1.5배 규칙 — 경계 공식

패턴 $Q1 - 1.5 \cdot IQR$ · $Q3 + 1.5 \cdot IQR$ 이상치 경계선 찾기

정의 가운데 절반 구간의 양쪽으로 IQR의 1.5배만큼 여유를 두고, 그 바깥을 이상치로 봅니다.

형태 $lower = Q1 - 1.5 \cdot IQR$ | $upper = Q3 + 1.5 \cdot IQR$

```
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
print(f"하한선: {lower}, 상한선: {upper}")
```

▶ 하한선: 141.875, 상한선: 338.875

왜 쓰나 변수 다섯 개만 채우면 끝납니다. $Q1 \rightarrow Q3 \rightarrow IQR \rightarrow lower \rightarrow upper$. 이 순서만 외우면 어떤 컬럼에도 적용할 수 있습니다.

응용 왜 하필 1.5일까요. 경험적으로 정한 균형점입니다. 1배로 좁히면 너무 예민한 경보기가 되어 정상값까지 속아 내고, 3배로 넓히면 너무 둔해져 진짜 이상치를 놓칩니다.

```
# 배수를 바꿔 가며 몇 건이 잡히는지
c = "사이클타임"
Q1, Q3 = df[c].quantile(0.25), df[c].quantile(0.75)
I = Q3 - Q1
for k in [1.0, 1.5, 2.0, 3.0]:
    m = (df[c] < Q1 - k*I) | (df[c] > Q3 + k*I)
    print(f"{k}배: {m.sum()}건 ({m.mean()*100, 1}%) 상한={round(Q3 + k*I, 1)}")
```

▶ 1.0배: 6건 (3.0%) 상한=51.0
▶ 1.5배: 6건 (3.0%) 상한=58.6
▶ 2.0배: 6건 (3.0%) 상한=66.2
▶ 3.0배: 5건 (2.5%) 상한=81.3

개념

이 컬럼에서는 1배부터 2배까지 결과가 6건으로 똑같습니다. 3배로 넓혀야 겨우 한 건이 빠집니다. 이상치들이 76.3·91.2·93.1·125.9·652.3·6170으로 경계에서 한참 떨어져 있기 때문입니다. 이렇게 배수를 바꿔도 결과가 안 변한다면 경계 근처에 애매한 값이 없다는 뜻이라 안심하고 처리해도 됩니다.

주의

박스플롯의 수염(**whisker**) 끝이 정확히 이 경계입니다. 수염 밖에 찍힌 점이 곧 $1.5 \times \text{IQR}$ 을 벗어난 값입니다. 다음 단원에서 그림을 그리게 되면 오늘 계산한 숫자와 그림이 일치하는지 확인해 보십시오. **두 결과가 다르면 코드나 그림 어느 한쪽이 틀린 것입니다.**

3-3. 실습 1 · 2 — mask로 골라내고 세기

Practice/16_02_example.py — 실습 1·2

```
CD = "data/16_diecasting.csv"
df = pd.read_csv(CD)

Q1 = df["사이클타임"].quantile(0.25)
Q3 = df["사이클타임"].quantile(0.75)
IQR = Q3 - Q1
print(round(Q1, 2), round(Q3, 2), round(IQR, 2))

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print(round(lower, 2), round(upper, 2))

# 하한보다 작거나 상한보다 큰 조건 — 각각 괄호로 감싸고 또는(|)로 연결
mask = (df["사이클타임"] < lower) | (df["사이클타임"] > upper)
print(df[mask][["샷", "사이클타임", "상태"]])
print(mask.sum(), round(mask.mean() * 100, 1))
```

```
> 20.8 35.92 15.12
> -1.89 58.61
>
>   샷 사이클타임 상태
> 146 147    125.9    0
> 180 181    652.3    1
> 181 182     93.1    1
> 190 191     91.2    1
> 195 196     76.3    1
> 199 200   6170.0    1
>
> 6 3.0
```

하한이 **-1.89**라는 점이 재미있습니다. 사이클타임이 음수일 수는 없으니 **하한 쪽 이상치는 원리적으로 나올 수 없습니다.** 이 컬럼의 이상치는 전부 위쪽입니다. 경계를 계산했으면 **그 값이 물리적으로 말이 되는지 한 번 보는 것이 좋습니다.**

개념

`mask.sum()`으로 개수, `mask.mean() * 100`으로 퍼센트. **어제 결측률을 구할 때 쓴 방법과 완전히 같습니다.**
참·거짓 표에 `sum`을 붙이면 개수, `mean`을 붙이면 비율. 대상이 결측에서 이상치로 바뀌었을 뿐 **도구는 그대로입니다.**

이상치 비율	판단	조치
1% 안팎	소수의 명백한 이상	지워도 무방 — 처리 안정적
3.0% (오늘)	정상 범위	제거·보정 중 선택
20~30%	처리 방법 재검토 필요	경계 설정을 다시 볼 것
너무 높음	경계 설정 자체를 의심	분포가 정규분포가 아닐 가능성

주의

비율을 함께 봐야 하는 이유는 처리 전에 규모를 파악하기 위해서입니다. 수술 전 검사와 같습니다. 3%면 지워도 큰 손실이 아니지만, **30%가 이상치로 잡혔다면 그것은 이상치가 아니라 경계가 잘못된 것입니다.** 한쪽으로 크게 치우친 분포에서 IQR을 쓰면 이런 일이 생깁니다.

3-4. IQR이 놓치는 것 — 실린더압력의 함정

IQR도 만능은 아닙니다. 그리고 **오늘 데이터에 그 한계를 정확히 보여주는 사례가 있습니다.**

상한 쪽이 하나도 안 잡히는 이유

```
c = "실린더압력"
Q1, Q3 = df[c].quantile(0.25), df[c].quantile(0.75)
I = Q3 - Q1
L, U = Q1 - 1.5*I, Q3 + 1.5*I

print("Q1:", Q1, "Q3:", Q3, "max:", df[c].max())
print("상한:", U, "→ 상한 초과:", (df[c] > U).sum(), "건")
print("하한:", L, "→ 하한 미만:", (df[c] < L).sum(), "건")

# 265가 몇 번 나오는가
print("265.0 인 행:", (df[c] == 265.0).sum(), "/ 유효값:", df[c].count())
```

- ▶ Q1: 215.75 Q3: 265.0 max: 265.0
- ▶ 상한: 338.875 → 상한 초과: 0 건
- ▶ 하한: 141.875 → 하한 미만: 6 건
- ▶ 265.0 인 행: 53 / 유효값: 188

Q3와 최댓값이 265.0으로 똑같습니다. 그리고 265.0인 행이 53건입니다. 유효값 188건 중 28%가 정확히 같은 값에 묻혀 있습니다.

설비 적용

이것은 **센서 상한 포화**의 전형적인 모습입니다. 측정 가능한 최댓값이 265라서 그보다 큰 값은 전부 265로 기록된 것으로 보입니다. 실제 압력은 270일 수도 300일 수도 있지만 데이터에는 265로만 남습니다. **진짜 이상 신호가 데이터에서 지워진 상태**입니다. 그런데 IQR은 이것을 전혀 잡지 못합니다. 상한 338.875가 최댓값 265보다 크니 **상한 초과가 0건**으로 나오는 것입니다.

한계	무슨 뜻	오늘 데이터의 예
단변량	한 컬럼씩만 봄	온도와 진동이 함께 튀는 결합 이상은 못 잡음
분포 가정	한쪽으로 치우친 데이터에 약함	실린더압력 265 포화 — 상한 0건
해석 불가	노이즈인지 신호인지 구분 못 함	125.9초가 상태 0인 이유를 모름

개념

그렇다고 IQR이 약한 도구라는 뜻은 아닙니다. 교안의 표현이 정확합니다. **"이상치 후보를 빠르게 찾아주는 똑똑한 조수. 후보를 짚어주고 최종 결정은 사람."** 한계를 알고 쓰면 든든한 출발점입니다. 265에 53건이 몰려 있다는 사실은 `value_counts()`로 따로 확인해야 보입니다. 도구 하나로 다 되는 일은 없습니다.

STEP 4

이상치 처리 세 가지

제거 · clip 보정 · NaN 변환 후 채움

탐색이 끝났으니 처리입니다. 무조건 지우기가 아니라 세 가지 선택지가 있고, 컬럼마다 다르게 고를 수 있습니다.

4-1. 세 방법 한눈에

방법	하는 일	코드	언제 쓰나
① 제거	이상치 행을 삭제	<code>df[~mask]</code>	명백한 노이즈 · 비율 낮을 때
② 보정	경계값으로 눌러 평탄화	<code>.clip(lower, upper)</code>	행을 살리며 극단만 누를 때
③ 변환	NaN으로 바꾼 뒤 채움	<code>.mask(m).fillna()</code>	주변값으로 자연스럽게 흡수
④ 보존	처리하지 않고 표시만	<code>df["의심"] = mask</code>	신호로 의심될 때

개념

네 번째 보존을 잊지 마십시오. 교안이 따로 한 장을 할애한 이유가 있습니다. 1장에서 배운 대로 **고장 신호로 의심되면 처리하지 않고 남깁니다**. 다만 그냥 두는 것이 아니라 **이상 의심 구간이라는 컬럼에 1로 기록해 둡니다**. 이 한 단계가 나중에 고장 분석의 출발점이 됩니다.

4-2. 제거 — 물결로 뒤집기

패턴 `df[~mask]` 이상치가 아닌 행만 남기기

정의 ~는 부정 연산자입니다. 참·거짓을 뒤집어 이상치가 아닌 행만 골라냅니다.

형태 `df_clean = df[~mask]`

```
mask = (df["사이클타임"] < lower) | (df["사이클타임"] > upper)
```

```
# 이상한 걸 제외한 나머지 멀쩡한 결과들
df_clean = df[~mask]
print(len(df), len(df_clean))
print("제거 후 평균:", df_clean["사이클타임"].mean())
```

▶ 202 196

▶ 제거 후 평균: 27.275824175824173

왜 쓰나 평균이 64.75에서 27.28로 내려왔습니다. 6건을 뺐을 뿐인데 평균이 절반 이하가 됐습니다. 6170초 하나의 무게가 그만큼 컸다는 뜻입니다.

응용 제출 코드에 **함정 하나가 숨어 있습니다**. 정상 범위를 고르는 방법이 두 가지인데, 결과가 다릅니다.

```
# 방법 1 - 이상치를 뒤집기
print("~mask :", len(df[~mask]))

# 방법 2 - 정상 범위를 직접 지정
mask_ok = (df["사이클타임"] >= lower) & (df["사이클타임"] <= upper)
print("mask_ok :", len(df[mask_ok]))

print("차이의 정체 — 사이클타임 NaN:", df["사이클타임"].isna().sum())
```

```
▶ ~mask      : 196
▶ mask_ok    : 182   ← 14건이 더 빠짐
▶
▶ 차이의 정체 — 사이클타임 NaN: 14
```

개념

결측 14건 때문입니다. NaN < lower도 거짓이고 NaN > upper도 거짓이라, mask는 결측 행을 **이상치가 아니라고 판정**합니다. 그래서 ~mask에는 결측 행이 **포함됩니다**. 반대로 mask_ok는 NaN >= lower가 거짓이라 결측 행을 **정상이 아니라고 판정**해 빼 버립니다.

자주 하는 실수

22일차에 배운 **NaN의 비교 연산 함정**이 여기서 다시 나타납니다. NaN은 어떤 비교에도 거짓을 돌려주므로, **부정하면 포함되고 긍정하면 배제됩니다**. 이 데이터에서 14건이나 차이 나므로 무시할 수 없습니다. **이상치를 걸러내기 전에 결측을 먼저 처리하는 것이 안전한 순서**인 이유입니다.

4-3. 보정 — clip으로 눌러 평탄화

메서드 `.clip()` 하한보다 작으면 하한으로, 상한보다 크면 상한으로

정의 경계 밖 값을 **경계값으로 강제로 끌어당깁니다**. 행을 버리지 않으므로 데이터 수가 그대로 유지됩니다. 윈저화(Winsorizing)라고도 부릅니다.

형태 `df["열"].clip(lower=하한, upper=상한)`


```
df["사이클타임_clipped"] = df["사이클타임"].clip(lower=lower, upper=upper)
print(df["사이클타임_clipped"].agg(["min", "max", "mean"]))
```

```
▶ min      20.600000
▶ max      58.612500 ← 6170이 58.61로 눌림
▶ mean     28.275931
▶ Name: 사이클타임_clipped, dtype: float64
```

왜 쓰나 제출 코드의 주석이 잘 요약했습니다. "시계열 신호나 추세가 깨지지 않고 데이터 수도 그대로 유지되는 현업 다빈도 기법." 행을 지우면 그 시점의 다른 센서값도 함께 사라지지만, clip은 그 컬럼만 손보고 나머지는 지킵니다.

응용 대가는 정보 손실입니다. 6170초와 652초가 둘 다 58.61로 똑같아집니다. 어느 쪽이 더 심각했는지가 사라집니다.

```
# 원본과 보정 후를 나란히 — 상위 3건
idx = df["사이클타임"].nlargest(3).index
print(df.loc[idx, ["샷", "사이클타임", "사이클타임_clipped"]])
```

```
▶ 샷 사이클타임 사이클타임_clipped
▶ 199 200 6170.0      58.6125
▶ 180 181 652.3      58.6125
▶ 146 147 125.9      58.6125
```

개념

평균을 비교해 보십시오. 원본 64.75 → 제거 27.28 → 보정 28.28. 제거와 보정 모두 평균을 중앙값 22.6 근처로 끌어내렸습니다. 다만 보정 쪽이 살짝 높은데, 이상치를 0으로 만든 것이 아니라 58.61이라는 값으로 남겨 뒀기 때문입니다.

주의

clip()은 한쪽만 쓸 수도 있습니다. clip(lower=0)은 음수만 0으로 올리고 위쪽은 건드리지 않습니다. 물리적으로 음수가 불가능한 컬럼에 유용합니다.

4-4. 변환 — NaN으로 바꾼 뒤 채우기

메서드 .mask() 조건이 참인 자리를 NaN으로 바꾸기

정의 조건에 해당하는 값을 NaN으로 갈아 끼웁니다. 그러면 지난 시간에 배운 fillna()를 그대로 쓸 수 있습니다.

형태 s.mask(조건).fillna(대체값)

```
s_masked = df["사이클타임"].mask(mask)
print("mask 후 NaN 개수:", s_masked.isna().sum())
```

```
# 중앙값을 계산할 때 NaN은 제외된다
s_fixed = s_masked.fillna(s_masked.median())
print("채운 뒤 평균:", s_fixed.mean())
```

```
▶ mask 후 NaN 개수: 20
▶ 채운 뒤 평균: 26.802970297029702
```

왜 쓰나 세 처리 중 **가장 부드러운 방식**입니다. 버리지도 않고 경계에 붙이지도 않고, **주변과 어울리는 값으로 대체**해 정상 분포 속에 자연스럽게 흡수시킵니다.

응용 NaN이 **20개**로 나온 것에 주목하십시오. 이상치는 6건이었는데 왜 20일까요.

```
print("이상치 :", mask.sum())
print("원래 결측 :", df["사이클타임"].isna().sum())
print("합계 :", mask.sum() + df["사이클타임"].isna().sum())
print("mask 후 NaN :", df["사이클타임"].mask(mask).isna().sum())
```

```
▶ 이상치 : 6
▶ 원래 결측 : 14
▶ 합계 : 20
▶ mask 후 NaN : 20
```

개념

이상치 6건 + 원래 있던 결측 14건 = 20건입니다. `mask()`가 새로 만든 결측과 원래 있던 결측이 합쳐진 것입니다. 그리고 이어지는 `fillna()`가 **둘을 구분하지 않고 한꺼번에 중앙값으로 채웁니다**. 결과적으로 결측 처리까지 함께 끝나는 셈인데, 이것이 의도한 동작인지는 확인이 필요합니다.

주의

왜 평균이 아니라 **중앙값**으로 채울까요. **평균은 이상치에 이미 오염됐을 수 있기 때문**입니다. 사이클타임 평균은 64.75인데 이 값에는 6170이 섞여 있습니다. 그 값으로 채우면 이상치를 지우려다 이상치를 심는 꼴이 됩니다. 중앙값 22.6은 안전합니다.

4-5. 실습 6 — 세 처리를 나란히 비교

실습 6은 실린더압력에 세 처리를 모두 적용해 평균 변화를 대조하는 문제입니다. 오늘 배운 것을 한 표에 모으는 자리입니다.

Practice/16_02_example.py — 실습 6

```
Q1 = df["실린더압력"].quantile(0.25)
Q3 = df["실린더압력"].quantile(0.75)
IQR = Q3 - Q1
L = Q1 - 1.5 * IQR
U = Q3 + 1.5 * IQR

m = (df["실린더압력"] < L) | (df["실린더압력"] > U)
채움 = df["실린더압력"].mask(m).fillna(df["실린더압력"].mask(m).median())

print(round(df["실린더압력"].mean(), 2))          # 처리 전
print(round(df[~m]["실린더압력"].mean(), 2))      # 제거
print(round(df["실린더압력"].clip(L, U).mean(), 2)) # 보정
print(round(채움.mean(), 2))                      # 중앙값 채움
```

- ▶ 234.31
- ▶ 238.39
- ▶ 235.31
- ▶ 238.05

처리	평균	중앙값(218) 대비	해석
처리 전	234.31	+16.31	아래쪽 이상치가 평균을 끌어내림
제거	238.39	+20.39	108·113짜리 6건이 빠져 평균이 올라감
보정	235.31	+17.31	경계 141.875로 높였을 뿐 완전히 빠진 않음
중앙값 채움	238.05	+20.05	제거와 비슷 — 중앙값이 218이라

주의

여기서 **처리 후 평균이 오히려 중앙값에서 멀어졌습니다**. 교안의 예시(온도 데이터)에서는 처리 후 평균이 중앙값에 가까워지는데, 이 데이터는 반대입니다. 이유는 **3-4에서 본 265 포화** 때문입니다. 265에 53건이 몰려 있어 중앙값 218과 평균 사이의 격차가 원래부터 위쪽으로 있었고, 아래쪽 이상치 6건을 빼니 그 격차가 더 벌어진 것입니다.

"평균이 중앙값에 가까워지면 처리 성공"이라는 신호가 항상 통하는 것은 아닙니다.

그리고 통계가 안정됐다고 무조건 좋은 처리도 아닙니다. **신호성 이상치였다면 정보를 잃은 것입니다.** 실린더압력 108·113은 상태가 전부 1로 기록되어 있습니다. **6건 모두 문제 있는** 샷입니다. 이것을 지우는 것은 고장 기록을 지우는 일입니다.

4-6. 어느 방법을 고를 것인가

상황	고를 방법	이유
명백한 노이즈 · 비율 5% 미만	제거	추측이 없어 가장 정직
행 수가 중요 · 다른 컬럼도 써야 함	보정 clip	행을 잃지 않고 극단만 누름
분포를 자연스럽게 유지하고 싶음	NaN 변환 후 채움	주변과 어울리는 값으로 흡수
고장 신호로 의심됨	보존 + 표시	지우면 고장 단서가 사라짐

주의

통계가 안정됐다고 무조건 좋은 처리는 아닙니다. 평균이 중앙값에 가까워지면 흔히 성공 신호로 읽지만, 그것이 **신호성 이상치를 지운 결과**라면 정보를 잃은 것입니다. 그리고 무언가 바꿨으면 **반드시 무엇을 어떻게 바꿨는지 기록**해야 합니다. 처리 후 결과가 이상할 때 되짚을 근거가 됩니다.

STEP 5

중복 데이터

deduplicated · keep · drop_duplicates

이상치가 **값 하나**의 문제라면, 중복은 **행 하나가 통째로 반복되는** 문제입니다. 마지막 정제 항목입니다.

5-1. 왜 중복이 문제인가

비유

교안의 비유가 명쾌합니다. **설문지에 같은 사람이 똑같은 답을 두 번 낸 것과** 같습니다. 한 명의 의견이 두 표로 반영되어 **결과가 한쪽으로 치우칩니다**. 새 정보는 하나도 없는데 통계만 왜곡되는 것입니다.

원인	상황	현장 의미
수집 중복	통신 재시도·시스템 재시작으로 같은 값이 두 번 저장	수집 장치 설정 점검
파일 병합	날짜·설비별 파일을 합칠 때 겹치는 구간	병합 규칙 재검토
가공 실수	조건 오류·복사 붙여넣기	스크립트 검토

개념

교안이 짙은 관점이 중요합니다. **중복은 데이터 자체의 문제라기보다 처리 과정의 문제**입니다. 지우는 데서 그치지 말고 **왜 생겼는지** 봐야 합니다. **중복이 있다는 것은 수집 과정에 문제가 있었다는 단서**입니다. 원인을 짚지 않으면 다음 달에 또 생깁니다.

5-2. 실습 7 — duplicated로 찾기

메서드 `.duplicated()` 앞에 똑같은 행이 있었는지 참·거짓으로

정의 행마다 **앞쪽에 똑같은 행이 있었는지**를 True/False로 표시합니다. **처음 나온 행은 False**, 두 번째부터 True입니다.

형태 `df.duplicated()` | `df.duplicated(keep=False)`

```
# 완전 중복된 행만 골라내기
print(df[df.duplicated()])
print("중복 개수:", df.duplicated().sum())
print("전체 행 수:", len(df))
```

```
▶   샷   실린더압력   주조압력   사이클타임   비스킷두께   형체력   상태
▶ 200     8   215.0   1038.0    20.9    11.0   258.0     0
▶ 201    89   235.0   1137.0    22.7    13.0   261.0     0
▶
▶ 중복 개수: 2
▶ 전체 행 수: 202
```

왜 쓰나 이상치 mask와 똑 닮은 True/False입니다. 이상치 탐색에서 익힌 마스크 개념이 그대로 적용됩니다. 그대로 출력하면 202줄이라 보기 어려우니 `df[...]`로 감싸 **중복 행만** 추려 봅니다.

응용 keep 옵션이 어느 것을 남기고 어느 것을 표시할지 정합니다. 이름과 동작이 일치해서 외우기 쉽습니다.

```
print("keep 기본(first):", df.duplicated().sum())
print("keep='last'      :", df.duplicated(keep="last").sum())
print("keep=False       :", df.duplicated(keep=False).sum())
```

```
▶ keep 기본(first): 2   ← 처음 것 살리고 이후만 표시
▶ keep='last'       : 2   ← 나중 것 살리고 앞의 것 표시
▶ keep=False        : 4   ← 겹친 행을 원본까지 모두 표시
```

개념

keep=False가 4로 나오는 것이 핵심입니다. 겹친 짝을 전부 표시하기 때문입니다. 8번 샷이 두 줄, 89번 샷이 두 줄이니 총 4줄입니다. 중복을 지우려면 first나 last를, 겹친 것을 전부 확인하려면 False를 씁니다.

주의

데이터를 받으면 `df.duplicated().sum()`을 습관처럼 실행하십시오. 짧지만 강력한 점검입니다. 0이면 깨끗, 0보다 크면 정리 필요, 전체의 절반이 나오면 같은 파일을 두 번 읽었을 가능성을 의심해야 합니다.

5-3. 실습 8 · 9 — 제거와 인덱스 정리

메서드 `.drop_duplicates()` 중복을 한 행만 남기고 제거

정의 중복된 행들을 한 행만 남기고 지웁니다. subset=으로 기준 컬럼을 지정할 수 있습니다.

형태 `df.drop_duplicates(subset=[...], keep="last")`

```
print(len(df))

# 완전 중복 제거
df_onlyone = df.drop_duplicates()
print(len(df_onlyone))

# subset으로 특정 컬럼만 기준 삼아 제거
df_onlyone_shot = df.drop_duplicates(subset=["샷"], keep="last")
print(len(df_onlyone_shot))
```

- ▶ 202
- ▶ 200 ← 2건 제거
- ▶ 200 ← 샷 번호 기준도 같은 결과

왜 쓰나 완전 중복은 모든 컬럼 값이 같은 행이라 하나만 남기면 됩니다. 부분 중복은 핵심 컬럼만 같은 행인데, 무엇이 같으면 같은 기록인지 사람이 정해 줘야 합니다.

응용 subset 사용에는 원칙이 있습니다. 컬럼을 적게 넣을수록 기준이 느슨해져 많이 묶입니다. 샷 번호 하나만 기준으로 삼으면 다른 값이 달라도 같은 기록으로 봅니다.

```
# 기준을 바꿔 가며 몇 행이 남는지
print("완전 중복      :", len(df.drop_duplicates()))
print("샷만 기준      :", len(df.drop_duplicates(subset=["샷"])))
print("샷+압력 두 개  :", len(df.drop_duplicates(subset=["샷", "실린더압력"])))
print("샷+압력+주조압력 :",
      len(df.drop_duplicates(subset=["샷", "실린더압력", "주조압력"])))
```

- ▶ 완전 중복 : 200
- ▶ 샷만 기준 : 200
- ▶ 샷+압력 두 개 : 200
- ▶ 샷+압력+주조압력 : 200

개념

이 데이터에서는 어떤 기준을 써도 **200행으로 같습니다**. 겹친 두 쌍이 **모든 컬럼이 완전히 같은 완전 중복**이기 때문입니다. 부분 중복이 섞인 데이터라면 기준에 따라 결과가 달라집니다.

자주 하는 실수

subset 기준이 틀리면 **멸절한 데이터를 지웁니다**. 같은 샷 번호가 서로 다른 날짜에 두 번 나올 수 있는 데이터라면, 샷만 기준으로 삼으면 하나가 억울하게 사라집니다. **기준 컬럼을 정하려면 설비를 이해해야** 합니다.

메서드 `.reset_index(drop=True)` 끊긴 인덱스를 0부터 다시 매기기

정의 행을 뺀 뒤 **등성등성 끊긴 인덱스**를 0부터 연속으로 다시 붙입니다. `drop=True`는 **옛 인덱스를 버린다**는 뜻입니다.

형태 `df.drop_duplicates().reset_index(drop=True)`

```
df_clean = df.drop_duplicates()
print("정리 전 인덱스:", df_clean.index.min(), "~", df_clean.index.max(), "· 길이",
      len(df_clean))
```

```
df_reset = df_clean.reset_index(drop=True)
print("정리 후 인덱스:", df_reset.index.min(), "~", df_reset.index.max(), "· 길이",
      len(df_reset))
```

▶ 정리 전 인덱스: 0 ~ 199 · 길이 200

▶ 정리 후 인덱스: 0 ~ 199 · 길이 200

왜 쓰나 행을 빼면 인덱스에 구멍이 납니다. 0, 1, 2, 5, 7처럼 끊기면 이후 `iloc`로 접근할 때 헛갈리고, 표를 출력해도 보기 나쁩니다.

응용 이 데이터에서는 **정리 전후가 똑같이 0~199**입니다. 지워진 두 행이 **맨 끝(200번·201번)**이었기 때문에 구멍이 안 났습니다. 중간이 지워지는 상황을 만들어 보면 차이가 보입니다.

```
# 중간 행을 지워 구멍을 만들어 보면
gap = df.drop(index=[1, 3, 5])
print("구멍 난 인덱스 :", gap.index[:6].tolist())
print("reset 후   :", gap.reset_index(drop=True).index[:6].tolist())
```

▶ 구멍 난 인덱스 : [0, 2, 4, 6, 7, 8]

▶ reset 후 : [0, 1, 2, 3, 4, 5]

자주 하는 실수

`drop=True`를 빼먹으면 **옛 인덱스가 index**라는 새 컬럼으로 추가됩니다. 쓸데없는 컬럼이 하나 늘어나니 거의 언제나 `drop=True`를 붙입니다. 어제 배운 `to_csv(index=False)`와 같은 맥락입니다.

개념

이상치 정리 + 중복 정리 + 인덱스 정리 = 정제 한 바퀴 완성. 행을 빼는 작업 뒤에는 인덱스 정리를 붙이는 습관을 들이십시오. 다음 장의 종합 실습이 정확히 이 순서로 진행됩니다.

STEP 6

용접 데이터 종합 정제

탐색 → 보정 → 중복 정리 → 저장

마지막 실습입니다. 오늘 배운 흐름을 완전히 다른 현장의 데이터에 그대로 적용해 봅니다. 다이캐스팅에서 저항 점용접으로 무대가 바뀝니다.

6-1. 저항 점용접 데이터

설비 적용

저항 점용접은 두 판재를 겹쳐 전극으로 누르고 큰 전류를 순간적으로 흘려 접합부를 녹여 붙이는 공정입니다. 자동차 차체 한 대에 수천 점이 들어갑니다. 전류가 너무 낮으면 판재가 안 붙고, 너무 높으면 구멍이 뚫립니다. 그래서 통전전류가 정상 범위를 벗어났는지가 품질과 직결됩니다.

컬럼	뜻	역할
용접전압 · 용접전류	설정한 용접 조건	입력 조건
통전전압 · 통전전류	실제로 흐른 전압 · 전류	실측값 — 오늘의 분석 대상
가압력	전극이 판재를 누르는 힘	접촉 저항에 영향
판정	1 정상 · 0 불량	정답 컬럼

6-2. 실습 10 — 전체 흐름 한 번에

Practice/16_02_example.py — 실습 10

```
WD = "data/16_welding.csv"
wf = pd.read_csv(WD)
c = "통전전류"

# ① 탐색 — IQR 경계로 이상치 개수·비율 확인
q1, q3 = wf[c].quantile(0.25), wf[c].quantile(0.75)
lo, hi = q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)
m = (wf[c] < lo) | (wf[c] > hi)
print(int(m.sum()), round(m.mean() * 100, 1))

# ② 보정 — clip으로 경계값에 눌러 담기
wf[c] = wf[c].clip(lower=lo, upper=hi)

# ③ 중복 제거 + 인덱스 정리
wf = wf.drop_duplicates().reset_index(drop=True)
print(len(wf))

# ④ 정제된 데이터를 파일로 저장
wf.to_csv("data/16_welding_cleaned.csv", index=False)

> 24 14.8
> 158
```

네 단계가 열 줄 남짓에 다 들어 있습니다. **탐색 → 보정 → 중복 정리 → 저장**. 오늘 배운 전부입니다. 162행이 158행으로 줄었으니 **중복이 4건** 있었습니다.

6-3. 이상치가 정말 불량과 맞아떨어지는가

제출 코드의 주석에 "**판정 0 불량과 대체로 일치**"라고 적혀 있습니다. 이 문장이 맞는지 직접 확인해 보겠습니다. 오늘 가장 흥미로운 결과가 여기서 나옵니다.

IQR 이상치와 실제 불량량의 관계

```
wf = pd.read_csv(WD)    # 보정 전 원본으로 다시
q1, q3 = wf[c].quantile(0.25), wf[c].quantile(0.75)
lo, hi = q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)
m = (wf[c] < lo) | (wf[c] > hi)

print("이상치    :", int(m.sum()))
print("불량(판정 0)  :", int((wf["판정"] == 0).sum()))
print("이상치이면서 불량:", int((m & (wf["판정"] == 0)).sum()))
```

▶ 이상치 : 24
▶ 불량(판정 0) : 20
▶ 이상치이면서 불량: 20

	판정 0 (불량)	판정 1 (정상)	합계
이상치로 잡힘	20	4	24
정상으로 통과	0	138	138
합계	20	142	162

설비 적용

불량 20건이 하나도 빠짐없이 전부 이상치로 잡혔습니다. 놓친 불량량이 0건입니다. 통계 용어로 **재현율(recall)** 100%입니다.

대신 **정상인데 이상치로 잡힌 것이 4건** 있습니다. 24건 중 20건이 진짜 불량이니 **정밀도(precision)** 83.3%입니다.

현장에서 이 정도면 **아주 좋은 결과**입니다. 품질 검사에서는 **불량을 놓치는 것이 정상을 잘못 잡는 것보다 훨씬 위험**합니다. 불량 하나가 고객에게 나가는 것과, 멀쩡한 것 네 개를 한 번 더 확인하는 것은 무게가 다릅니다.

그리고 이 결과는 오늘 배운 IQR이 **아무 근거 없는 기계적 규칙이 아니라는 증거**이기도 합니다. 판정 컬럼을 전혀 보지 않고 통전전류 하나만 가지고 계산했는데, 결과가 실제 품질 판정과 거의 일치했습니다.

주의

다만 **이 데이터에서 잘 맞았다는 것뿐**입니다. 통전전류가 불량과 직결되는 공정이라 잘 맞은 것이고, 다른 공정에서는 다를 수 있습니다. 그리고 **정답 컬럼이 있어서 확인할 수 있었다**는 점도 기억해야 합니다. 현실의 대부분 데이터에는 정답이 없어서 **잘 맞는지 알 방법이 없습니다**. 그래서 1장에서 배운 대로 현장의 판단이 필요합니다.

마지막으로 왜 제거가 아니라 보정(`clip`)을 골랐는지 생각해 보십시오. 24건을 지우면 불량 20건이 통째로 사라집니다. 불량을 예측하는 모델을 만들려는데 불량 데이터를 다 버리는 셈입니다. 1장의 화재경보기 이야기가 정확히 이 상황입니다. `clip`은 행을 살리면서 극단값만 누르므로 불량 20건이 데이터에 남습니다.

6-4. 오늘 배운 흐름의 완성형

단계	도구	이 실습에서	확인할 것
① 탐색	<code>quantile → IQR → mask</code>	24건 · 14.8%	비율이 20%를 넘지 않는가
② 판단	사람	전류 이상 = 불량 신호	노이즈인가 신호인가
③ 처리	<code>clip</code>	제거가 아닌 보정	불량 20건을 지키기 위해
④ 중복	<code>drop_duplicates + reset_index</code>	162 → 158행	왜 4건이 겹쳤는가
⑤ 저장	<code>to_csv(index=False)</code>	새 이름으로	다시 읽어 검증했는가

②번만 코드가 없습니다. 그리고 ②번이 ③번을 결정합니다. "전류 이상은 불량 신호다"라고 판단했기 때문에 제거가 아니라 보정을 고른 것입니다. 판단이 달랐다면 코드도 달라졌을 것입니다.

치트시트

오늘 나온 모든 도구를 한 표에

A-1. 분포 읽기

도구	문법	돌려주는 것
<code>.min()</code> · <code>.max()</code>	<code>df["열"].max() - df["열"].min()</code>	범위 — 이상치에 약함
<code>.quantile()</code>	<code>s.quantile(0.25)</code>	25% 지점 값 (Q1)
<code>.quantile([...])</code>	<code>s.quantile([0.25, 0.5, 0.75])</code>	세 값을 한 번에 (Series)
<code>.median()</code>	<code>s.median()</code>	중앙값 = Q2
<code>.describe().T</code>	<code>df[cols].describe().T</code>	컬럼이 행인 분포 요약표
평균-중앙값 격차	<code>(r["mean"] - r["50%"]).abs()</code>	이상치가 있다는 신호

A-2. 이상치 탐지와 처리

도구	문법	하는 일
IQR	<code>Q3 - Q1</code>	가운데 절반의 폭
경계	<code>Q1 - 1.5*IQR / Q3 + 1.5*IQR</code>	하한 · 상한
mask	<code>(s < lo) (s > hi)</code>	이상치 참·거짓 표
개수·비율	<code>mask.sum() / mask.mean()*100</code>	몇 건 · 몇 %
① 제거	<code>df[~mask]</code>	이상치 아닌 행만
② 보정	<code>s.clip(lower=lo, upper=hi)</code>	경계값으로 눌러 평탄화
③ 변환	<code>s.mask(m).fillna(s.median())</code>	NaN으로 바꾼 뒤 채움
④ 보존	<code>df["이상의심"] = mask</code>	처리하지 않고 표시만

A-3. 중복 처리

도구	문법	하는 일	이 데이터
<code>.duplicated()</code>	<code>df.duplicated().sum()</code>	중복 개수 (처음 것 제외)	2건
<code>keep="last"</code>	<code>df.duplicated(keep="last")</code>	나중 것을 살림	2건
<code>keep=False</code>	<code>df.duplicated(keep=False)</code>	겹친 행을 원본까지 모두	4건
<code>.drop_duplicates()</code>	<code>df.drop_duplicates()</code>	완전 중복 제거	202 → 200행
<code>subset=</code>	<code>drop_duplicates(subset=["샷"])</code>	기준 컬럼만 보고 제거	200행
<code>.reset_index()</code>	<code>.reset_index(drop=True)</code>	인덱스 0부터 다시	0~199

A-4. 복사해 쓰는 정제 골격

데이터를 받으면 이 순서로

```
import pandas as pd
```

```
df = pd.read_csv("파일.csv", encoding="utf-8")
```

```
c = "대상컬럼"
```

```
# 1) 탐색 — IQR 경계와 이상치 개수·비율
```

```
q1, q3 = df[c].quantile(0.25), df[c].quantile(0.75)
```

```
iqr = q3 - q1
```

```
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
```

```
m = (df[c] < lo) | (df[c] > hi)
```

```
print(f"경계 {lo:.2f} ~ {hi:.2f}")
```

```
print(f"이상치 {int(m.sum())}건 ({m.mean()*100:.1f}%)"
```

```
print(df[m])
```

```
# ← 눈으로 확인하는 단계, 생략 금지
```

```
# 2) 판단 — 노이즈인가 신호인가 (코드로 못 함)
```

```
# 3) 처리 — 셋 중 하나를 고른다
```

```
clean = df[~m]
```

```
# 제거
```

```
df[c] = df[c].clip(lower=lo, upper=hi)
```

```
# 보정
```

```
df[c] = df[c].mask(m).fillna(df[c].median())
```

```
# 변환
```

```
# 4) 중복 — 제거하고 인덱스 정리
```

```
print("중복:", df.duplicated().sum())
```

```
df = df.drop_duplicates().reset_index(drop=True)
```

```
# 5) 검증 후 새 이름으로 저장
```

```
df.to_csv("파일_clean.csv", index=False, encoding="utf-8")
```

주의

2번 단계에 코드가 없는 것이 이 골격의 핵심입니다. 1번에서 `print(df[m])`으로 이상치를 눈으로 보고, 그 값들이 무엇인지 판단한 다음에 3번에서 방법을 고릅니다. 판단을 건너뛰고 1번에서 3번으로 바로 가면 **고장 기록을 지우게 될 수도** 있습니다.

A-5. 오늘의 판단 기준

질문	기준	다음 행동
이상치인가	$Q1 - 1.5 \times IQR$ 미만 또는 $Q3 + 1.5 \times IQR$ 초과	후보로 표시 후 사람이 판단
지울까 남길까	물리적으로 가능한 값인가	불가능하면 노이즈 · 가능하면 신호
비율이 적정한가	20%를 넘는가	넘으면 경계 설정 자체를 재검토
어느 처리를 고를까	그 행의 다른 컬럼도 써야 하는가	써야 하면 clip · 아니면 제거
중복이 왜 생겼나	수집 · 병합 · 가공 중 어느 단계 인가	원인을 짚어야 재발을 막음

개념

다섯 기준 중 **두 번째만 코드로 답할 수 없습니다.** "물리적으로 가능한 값인가"는 데이터가 아니라 **설비를 아는 사람이** 답합니다. 오늘 배운 코드가 아무리 많아도 이 한 칸은 비어 있습니다. 그 칸을 채우는 것이 현장과의 대화입니다.

오류·함정 사전

제출 코드 교정 · 자주 걸리는 함정

B-1. 제출 코드 교정

오늘 실습 코드는 문법 오류 없이 정상 실행되었습니다. 다만 손볼 곳이 세 군데 있습니다.

① 내장 함수 이름을 변수로 덮어씀

문제 코드

```
# x 이렇게 쓰면
range = df["실린더압력"].max() - df["실린더압력"].min()
print(range)
```

```
# 같은 스크립트 뒤쪽에서 이런 코드를 만나면
for i in range(3):
    print(i)
```

```
▶ 157.0
▶
▶ TypeError: 'numpy.float64' object is not callable
```

바로잡기

```
# ✓ 다른 이름을 쓴다
value_range = df["실린더압력"].max() - df["실린더압력"].min()
print(value_range)
```

```
▶ 157.0
```

range는 파이썬 내장 함수입니다. 변수 이름으로 쓰면 그 이름이 덮어써워져 이후 반복문에서 range()를 쓸 수 없게 됩니다. 이 스크립트에서는 뒤에 range()를 쓰지 않아 오류가 안 났을 뿐입니다.

자주 하는 실수

같은 부류로 조심할 이름들이 있습니다. list, dict, set, str, int, sum, max, min, type, id, input, open, filter, map. 전부 내장 함수·자료형 이름입니다. 에디터에서 색이 다르게 표시되는 이름은 변수로 쓰지 않는다고 기억하십시오.

② 한글 변수명

구분	코드	문제
✗ 제출	정상 = df[~mask] 보정 = ... 채움 = ...	동작은 하지만 실무 비권장
✓ 교정	df_normal s_clipped s_filled	협업·이식성·인코딩 안전

파이썬 3은 한글 변수명을 허용하므로 **오류는 나지 않습니다**. 다만 팀 코드에 섞이면 곤란합니다. 인코딩이 다른 환경에서 깨질 수 있고, 라이브러리·프레임워크와 섞었을 때 자동완성이나 검색이 불편해집니다. **학습용으로 잠깐 쓰는 것은 괜찮지만 제출·배포 코드에는 영문을 쓰십시오**.

③ 주석의 이상치 목록이 4건으로 적혀 있음

주석과 실제의 불일치

주석에 적힌 목록 (4건)

```
#          샷 사이클타임 상태
# 146   147   125.9    0
# 180   181   652.3    1
# 181   182    93.1    1
# 190   191    91.2    1
```

실제 출력 (6건)

```
# 195   196    76.3    1  ← 빠져 있음
# 199   200  6170.0    1  ← 빠져 있음
```

▶ `mask.sum()` 은 6 으로 정확한데, 주석의 목록만 4줄

`mask.sum()` 이 6으로 정확히 나왔으니 **코드는 맞습니다**. 주석에 목록을 옮겨 적을 때 두 줄이 빠진 것으로 보입니다. **가장 중요한 6170초짜리 행이 빠졌다는 점**이 아쉽습니다. 출력을 주석으로 옮길 때는 **개수를 세어 확인**하는 습관이 필요합니다.

B-2. 오늘 걸리기 쉬운 함정 여섯 가지

✗ 이렇게 하면	무슨 일이	✓ 바로잡기
<code>~mask</code> 와 <code>mask_ok</code>	결측 때문에 결과가 14건 다름	결측을 먼저 처리하고 이상치 판정
<code>df_q.iloc[0.75]</code>	인덱스가 라벨이라 오류	<code>df_q.loc[0.75]</code>
<code>quantile(25)</code>	0~1 사이 소수가 아니라 오류	<code>quantile(0.25)</code>
<code>df.clip(...)</code>	저장 안 하면 원본은 그대로	<code>df[c] = df[c].clip(...)</code>
<code>mask()</code> 후 <code>fillna(mean)</code>	오염된 평균으로 채움	<code>fillna(median)</code>

× 이렇게 하면	무슨 일이	✓ 바로잡기
<code>reset_index()</code> — drop 없이	옛 인덱스가 index 컬럼으로 추가	<code>reset_index(drop=True)</code>

주의

첫 줄이 오늘 가장 조용하고 위험한 함정입니다. ~mask는 196행, mask_ok는 182행. **14건 차이가 나는데 둘 다 오류 없이 실행됩니다.** 어느 쪽이 맞는지는 목적에 따라 다릅니다. **결측 행도 남기고 싶다면 ~mask**, 값이 확실한 행만 쓰고 싶다면 **mask_ok**입니다. **중요한 것은 차이가 난다는 사실 자체를 아는 것입니다.**

B-3. 교안 예상 결과 대조

교안	교안이 말한 것	실제 실행 결과
16_01 p48	실습 2 — 실린더압력 최소 108 · 최대 265 · 범위 157	일치
16_01 p50	실습 4 — 평균 64.75 vs 중앙값 22.6, 정상만 27.67	일치
16_01 p51	실습 5 — Q1 215.75 · Q2 218 · Q3 265	일치
16_01 p53	실습 7 — 폭 49.25 · 15.12 · 6	일치
16_02 p56	실습 1 — IQR 15.12 · 하한 -1.9 · 상한 58.6	일치
16_02 p57	실습 2 — 이상치 6건, 비율 3.0%	일치
16_02 p59	실습 4 — 202행 → 196행, 평균 27.28	일치
16_02 p63	실습 8 — "202행 → 200행, subset 기준도 200행"	일치
16_02 p65	실습 10 — 이상치 24건(14.8%) · 정제 후 158행	일치

개념

오늘은 교안 예상 결과가 전부 맞습니다. 지난 회차들에서 라벨이 어긋나거나 다른 데이터 기준으로 적힌 경우가 잦았던 것과 대조적입니다. 그렇다고 확인을 생략하면 안 됩니다. **맞는 것을 확인하는 데도 같은 시간이 듭니다.** 그리고 틀린 곳이 한 군데라도 있으면 전부 다시 봐야 하니, 애초에 다 확인해 두는 편이 빠릅니다.

실습 하이라이트와 다음 시간

핵심 결론 정리 · 시각화 단원 예고

C-1. 오늘 얻은 여섯 가지 결론

#	결론	근거
1	이상치는 지울까가 아니라 노이즈인가 신호인가를 먼저 묻습니다.	고장은 언제나 이상치 — 화재경보기 비유
2	평균-중앙값 격차 자체가 이상치의 신호입니다.	사이클타임 평균 64.75 vs 중앙값 22.6
3	IQR이 못 잡는 이상이 있습니다.	실린더압력 265에 53건 포화 — 상한 초과 0건
4	부정과 긍정 조건은 결측에서 갈립니다.	~mask 196행 vs mask_ok 182행 (14건)
5	IQR이 불량 20건을 하나도 놓치지 않았습니다.	용접 통전전류 — 재현율 100%, 정밀도 83.3%
6	제거가 아니라 보정을 고른 데는 이유가 있습니다.	24건을 지우면 불량 20건이 통째로 사라짐

설비 적용

3번과 5번을 나란히 놓으면 IQR의 성격이 정확히 보입니다. **잘 맞을 때는 불량을 하나도 안 놓칠 만큼 잘 맞고, 안 맞을 때는 53건이 몰려 있는 이상 신호를 통째로 못 봅니다.** 도구를 믿되 결과를 그대로 받아들이지 말고 한 번 더 **확인하는 이유**가 여기 있습니다. `value_counts()`나 `describe()`로 교차 확인하는 습관이 그 안전장치입니다.

C-2. 오늘 푼 실습 정리

교안	실습	핵심 결과
16_01	2. 최소·최대·범위	실린더압력 108 ~ 265, 범위 157
16_01	3. 정렬해서 이상치 후보	6170·652·125.9초 — 상위 6건 중 5건이 상태 1
16_01	4. 평균·중앙값 비교	64.75 vs 22.6 · 정상만 27.67 — 20건이 37초를 끌어올림
16_01	5. quantile로 Q1·Q2·Q3	Q2 = 중앙값 확인 (218.0)

교안	실습	핵심 결과
16_01	6. describe 격차 큰 컬럼	주조압력 188.30 · 상대격차로는 사이클타임 186.5%
16_01	7. 가운데 절반 폭	49.25 · 15.12 · 6.00 — 다음 시간 IQR의 예고
16_02	1·2. IQR 경계와 개수	IQR 15.12 · 하한 -1.89 · 이상치 6건 3.0%
16_02	4·5. 제거와 clip 보정	제거 후 27.28 · 보정 후 28.28
16_02	6. 세 처리 비교	234.31 → 238.39 / 235.31 / 238.05
16_02	7~9. 중복과 인덱스	완전 중복 2건 · keep=False 4건 · 200행
16_02	10. 용접 종합 정제	이상치 24건(14.8%) · 정제 후 158행 저장

교안 16_02의 실습 3(박스플롯)은 하지 않았습니다. 시각화 도구를 아직 안 배웠기 때문입니다. 실습 코드에도 "다음 단원에서 다룰 matplotlib 라이브러리 사용 예제라 건너뛴"이라고 정확히 적혀 있습니다.

C-3. 다음 시간 예고 — 데이터 시각화

전처리 단원이 끝나고 **시각화 단원**이 시작됩니다. 오늘 숫자로 구한 것을 다음 시간에 그림으로 그립니다.

교안	주제	오늘과의 연결
17_01	선 그래프와 그래프 완성	시간에 따른 변화 · 제목 · 축 · 범례 · 저장
17_02	분포 · 범주 · 관계 시각화	막대 · 히스토그램 · 산점도 · subplot
18_01	Seaborn 분포와 범주	박스플롯 — 오늘 계산한 IQR을 그림으로
18_02	관계 분석과 시각화 리포트	상관 히트맵 — 21일차 상관 행렬을 그림으로

연결

박스플롯의 네 부분이 전부 오늘 계산한 것입니다. 상자가 Q1~Q3, 상자 속 선이 중앙값, 수염 끝이 $1.5 \times \text{IQR}$ 경계, 수염 밖 점이 이상치입니다. 오늘 손으로 구한 사이클타임 이상치 6건이 **수염 밖에 점 6개로 찍히는지 확인**해 보십시오. 교안의 표현대로 **"숫자와 그림은 서로를 검증합니다."** 두 결과가 다르면 어느 한쪽이 틀린 것입니다.

그리고 다음 시간 첫 관문은 **한글 폰트 설정**입니다. `plt.rcParams["font.family"] = "Malgun Gothic"`을 코드 맨 위에 붙이지 않으면 한글이 네모로 깨집니다. **코드 오류가 아니라 글꼴 설정 문제**이니 당황하지 마십시오.

C-4. 복습 순서

순서	무엇을	시간
1	Practice/16_02_example.py를 처음부터 다시 실행하며 출력 확인	20분
2	부록 A-4 골격을 16_welding.csv의 가압력 컬럼에 적용해 보기	15분
3	4-2의 ~mask 대 mask_ok 차이를 직접 실행해 14건을 눈으로 확인	10분
4	3-4의 실린더압력 265 포화를 value_counts()로 직접 확인	10분
5	6-3의 이상치 대 판정 교차표를 직접 만들어 보기	15분

C-5. 스스로 확인하는 질문 여섯 개

#	질문	확인 지점
1	노이즈와 신호를 가르는 첫 잣대는	물리적으로 가능한 값인가 — 1-2
2	IQR은 왜 이상치에 안 흔들리니까	가운데 절반만 보기 때문 — 3-1
3	왜 하필 1.5배입니까	경험적 균형점 — 1배는 예민, 3배는 둔감 — 3-2
4	~mask와 mask_ok가 다른 이유는	NaN이 모든 비교에 거짓 — 4-2
5	keep=False가 4를 주는 이유는	겹친 짝을 원본까지 모두 표시 — 5-2
6	용접 실습에서 제거가 아니라 clip을 쓴 이유는	불량 20건을 지우게 되므로 — 6-3

오늘의 성취

지을 것과 남길 것

연결

앞 표의 6번이 오늘 배운 것의 요약입니다. 도구를 아는 것과 **그 도구를 언제 쓸지 아는 것**은 다릅니다. clip을 쓸 줄 아는 것은 5분이면 배우지만, **왜 여기서 제거가 아니라 clip인지** 설명할 수 있으려면 오늘 배운 전부가 필요합니다.

오늘 배운 도구는 여덟 개입니다. `quantile`, `describe()`, `T`, `clip`, `mask`, `duplicated`, `drop_duplicates`, `reset_index`, 그리고 IQR 공식. 그런데 **도구 목록보다 중요한 것이 하나 있습니다.**

개념

지울지 말지가 아니라, 이것이 무엇인지를 먼저 묻는 것.

이상치를 만나면 노이즈인지 신호인지 묻고, 중복을 만나면 왜 겹쳤는지 묻습니다. 지난 시간에 결측을 만났을 때 왜 비었는지 물었던 것과 **같은 질문**입니다. 그리고 셋 다 **코드가 답해 주지 않습니다.**

강사님이 화재경보기 이야기를 길게 하신 이유가 이것입니다. **경보를 끄는 것은 쉽습니다. 경보가 왜 울렸는지 알아내는 것이 일입니다.**

한 걸음 더 나아가면, 오늘 배운 것은 **예지보전의 출발점**입니다. 고장을 미리 알아채려면 고장 데이터가 있어야 하는데, 고장은 언제나 이상치의 모습으로 나타납니다. **이상치를 다 지운 데이터로 학습한 모델은 고장을 한 번도 본 적이 없는 모델**입니다. 반대로 노이즈를 그냥 두면 모델이 **센서 오류를 학습**합니다. 그 사이에서 균형을 잡는 것이 전처리입니다.

지난 시간까지	오늘 더한 것	다음에 만날 것
빈 칸을 메우기	튀는 값과 겹친 행을 다루기	숫자를 그림으로 그리기
<code>dropna</code> · <code>fillna</code>	<code>quantile</code> · IQR · <code>clip</code> · <code>duplicated</code>	<code>matplotlib</code> · 박스플롯
결측치 단원	이상치 단원 — 전처리 완성	시각화 단원

마지막으로 오늘 확인한 숫자 하나만 다시 짚겠습니다. **용접 데이터에서 IQR이 잡은 이상치 24건 안에 불량 20건이 전부 들어 있었습니다.** 판정 컬럼을 한 번도 보지 않고 통전전류 하나만으로 계산한 결과입니다. **분포를 읽는 것만으로 품질을 짚어 낼 수 있다**는 뜻이고, 이것이 오늘 배운 도구들이 실제로 쓸모 있는 이유입니다.

— 24일차 정리 끝. 다음은 17_01 선 그래프부터 시작하는 데이터 시각화입니다.